

Open in app ↗

Medium

Search

Write

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)

Recurrent Neural Network: Maths in 5 Minutes



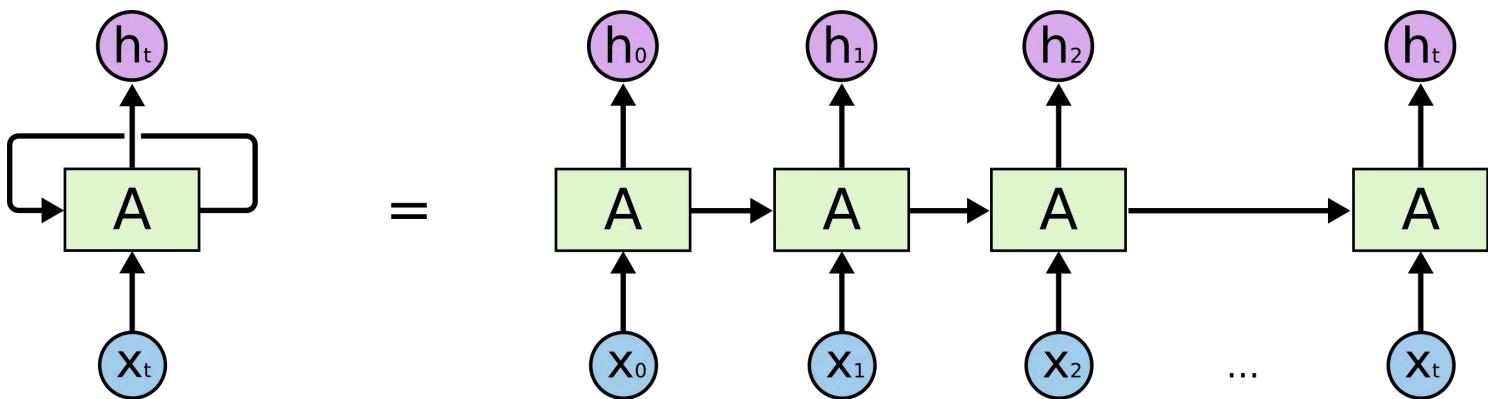
Vidisha Jitani

Follow

5 min read · Sep 21, 2020

199

1



So, there are tons of content on neural networks but, rarely are these focused on the maths. Even if there is such content, it's so much complex that we just leave it. And I believe, without that deeper understanding of how maths is actually working, it's really difficult to get the proper intuition and it will always be a magical black box to us.

Nature of the article

This is my third blog in the series. In case you haven't checked my previous articles I would highly recommend doing so since I'll build concepts on top of those. As usual, will keep it to the point and will just focus on the basic maths. For getting more depth, will definitely point to you to some good resources.

Sequence!

Just memorize this word! Whenever you think there is a dependency between different inputs and the sequencing of inputs matters — it has to be RNN. So what does it do extra than normal neural networks?

For example:

Inputs: ["This", "is", "a", "dog", "It", _].

Expected Output: "Barks"

Explanation —We had 5 inputs and the answer was found out on the basis of the 4th input, right?! So, somehow RNN might be sending the previous input's output to the next input. How many ways you can think of this? Pause for a sec! Think about it?

Being a software engineer, I was able to think of 2 ways. Let me know in the comments if you were able to think of any other as well.

1. **Back feeding based:** A neural network with 2 inputs — one for newer input and the other is the output of the previous input.

2. **Global State-based:** A global state being updated by all the inputs as and when they occur.

JFYI — First is your basic RNN and second is your LSTM. You just figured out the basic concept behind RNNs on your own. Yay!!

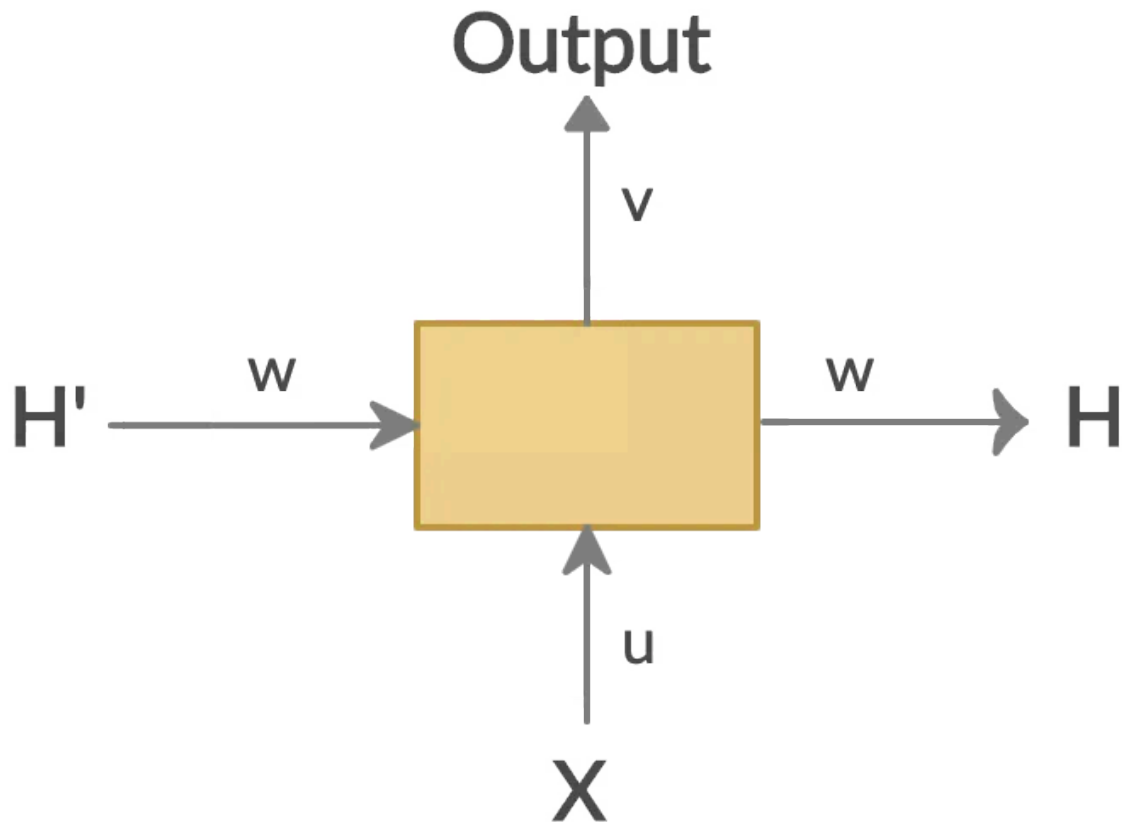
The Future of Humanity is Genetic Engineering and Neural Implants | Data Driven Investor

SXSW the leading tech, music, and film festival is on March 13-22nd 2020. It will feature talks on cutting edge...

www.datadriveninvestor.com

Forward Propagation Maths

Now, I have taken a really simple one layer of input for ease of calculations and understanding.



X — Input vector

Output — Predicted Output vector

H' — Previous Input's output to be fed in the current input

H — Current input's output to be fed in the next input

$u/v/w$ — Weights in the respective location

Read deeper.
Feel better.
Get access to more Medium.

Upgrade today

So, let's try to figure out the formula here. This is just simple maths. H is the summation of X and H' . Whereas Y is just H with a weight.

$$H = uX + wH'$$
$$\text{Output} = vH$$

That's all! See nothing scary till now! For simplicity, I'm not adding any activation functions logic here.

Back Propagation Maths

So, as we have previously mentioned, why do we calculate backpropagation? Basically, it tells us how much should we tune our variables to reduce the error between predicted output and expected Y .

$$E = \frac{(\text{Output} - Y)^2}{2}$$

$$\frac{\partial E}{\partial u} = ?, \quad \frac{\partial E}{\partial w} = ?, \quad \frac{\partial E}{\partial v} = ?$$

So, just by using chain rules and differential calculus (Already explained in my previous blogs)

$$\begin{aligned} H &= uX + wH' \\ \text{Output} &= vH \end{aligned} \quad (1)$$

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial \text{Output}} \frac{\partial \text{Output}}{\partial H} \frac{\partial H}{\partial w} = (\text{Output} - Y) (v) (H') \quad (2)$$

$$\frac{\partial E}{\partial v} = \frac{\partial E}{\partial \text{Output}} \frac{\partial \text{Output}}{\partial v} = (\text{Output} - Y) \cdot (H) = (\text{Output} - Y) \cdot (uX + wH') \quad (3)$$

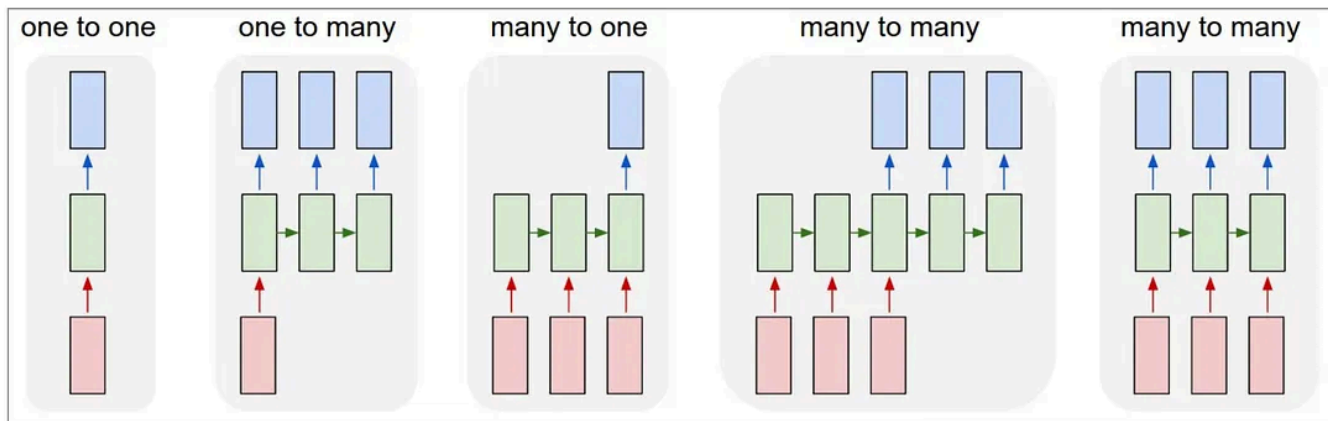
$$\frac{\partial E}{\partial u} = \frac{\partial E}{\partial \text{Output}} \frac{\partial \text{Output}}{\partial H} \frac{\partial H}{\partial u} = (\text{Output} - Y) \cdot u \cdot X \quad (4)$$

Yay! So these are the new values for u, v and w

$$\therefore \quad w_{\text{new}} = w_{\text{old}} - \frac{\partial E}{\partial w}, \quad v_{\text{new}} = v_{\text{old}} - \frac{\partial E}{\partial v}, \quad u_{\text{new}} = u_{\text{old}} - \frac{\partial E}{\partial u}$$

Vanishing And Exploding Gradient Problems

For simplicity of calculations, I had taken only one layer. But usually, we have a large number of such layers stacked one after another.



Now, let's focus on one formula from above

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial \text{Output}} \frac{\partial \text{Output}}{\partial H} \frac{\partial H}{\partial w} = (\text{Output} - Y) (v) (H')$$

As you can see, w 's tuning is dependent on previous layer's H' . Because of this 2 scenarios can occur:

1. $H' < 1$ — When you multiply with <1 , you get even a smaller number. As you keep moving backwards in the layer while back propagating, the value will keep on decreasing so much that it will almost tend to zero. Without any deltas, we won't be able to change w in the backward layers. And thus tuning it will become a major problem for us. This is called **Vanishing Gradient** problem.
2. $H' > 1$ — When you multiply with >1 , you get even a larger number. As you keep moving backwards in the layer while back propagating, the value will keep on increasing so much that it will almost tend to infinite. Because of which we will not be able to reach local minima of errors and

will keep on jumping in gradient descent graph. This is called **Exploding Gradient** problem.

So, due to these reasons RNNs are not ideal in cases where we have a huge dependent inputs.

Hope you likes the article. Please don't forget to show some love :)

Gain Access to Expert View — [Subscribe to DDI Intel](#)

Neural Networks

Data Science

Deep Learning

Machine Learning

Artificial Intelligence



Written by Vidisha Jitani

121 followers · 36 following

Metamorphosis 🦋

Follow

Responses (1)



Nitesh Kumar varma